Agentic RAG System for Macroeconomic Data Analysis

A Final Technical Report Submitted in Partial Fulfillment of the Requirements for DS 5500: Capstone — Applications in Data Science

Authors: Churou Deng

Team: 7

Instructor: Dr. Fatema Nafa

Course: DS 5500 — Capstone: Applications in Data Science

Institution: Northeastern University, Khoury College of Computer Sciences

Date: April 16, 2026

GitHub Repo: https://github.com/dcr225619/DS5500-RAG

Abstract

Background: RAG has been widely used in many domains. However, relatively limited work has explored RAG systems that are grounded in economic data, particularly macroeconomic indicators. And most existing economic RAG applications focus on economic knowledge bases or document-level question answering, rather than real-time time-series data query and analysis.
Objective: The primary goal of this project is to build an RAG system grounded in real-time economic data, capable of accurately retrieving relevant indicators, providing reliable economic trend analysis given a user's natural language query, and also providing an interpretable analysis of macroeconomic indicators for users with limited financial knowledge.
Methods: LLaMA, GPT, FRED database, RAG
Results: On the retrieval side, starting from the baseline retrieval F1 score of 0.785, both FAISS and the date parser improve retrieval performance by 15.9% and 13.7%, respectively. The full enhanced system achieves a 19.75% retrieval improvement, showing a cumulative effect. On the summarization side, the self-checks contribute the most, improving the retrieval performance by 9.0%. And the full pipeline achieves a 10.04% cumulative improvement over the baseline. In general, each component has a distinct and positive effect, semantic retrieval matters most for retrieval quality, and self-check mechanisms matter most for summary quality.
Conclusion: This project presents an agentic RAG system that makes macroeconomic data from FRED accessible and interpretable to general users. Through three iterative enhancements, FAISS semantic retrieval, a rule-based date parser, and a three-step self-check mechanism, the system achieves strong retrieval accuracy and low hallucination rates, even with a smaller open-source model like LLaMA. The few-shot prompting strategy further demonstrates that high-quality analysis is achievable without the cost of large proprietary models or expensive fine-tuning.

Keywords: LLaMA, GPT, FRED, Agentic RAG, time-series data analysis

Contents

1 Introduction **5**

1.1 Problem Statement and Motivation 5

1.2 Research Questions / Objectives 5

1.3 Report Structure 6

2 Related Work **6**

2.1 Existing Approaches to the Problem 6

2.2 Relevant Model Architectures 6

2.3 Datasets and Benchmarks 7

2.4 Positioning of This Work 7

3 Data Pipeline and Preprocessing **7**

3.1 FRED Data Repository Description 7

3.2 Fine-tuning/Test Datasets 8

3.3 Time Series Data Analysis 9

3.4 Series Index Guide Generation 10

4 Model Selection and Implementation **11**

4.1 Model Family Justification 11

4.2 Model Architecture 12

4.3 Coding Paradigm and Design Architecture 14

4.3.1 Paradigm Choice 14

4.3.2 Module Structure 14

4.4 Hyperparameters 15

4.5 Key Components 15

4.5.1 Prompt Engineering 15

4.5.2 Rule-Based Date Parser 16

4.5.3 Three-step Self-check 16

4.6 Fine-tuning . . . 18
4.7 Why Compare Multiple Models and Variants . . . 19

5 Results and Model Evaluation **19**
5.1 Evaluation Metrics . . . 19
5.2 Retrieval Accuracy Evaluation . . . 19
5.3 Summary Quality Evaluation . . . 20
5.4 Error Analysis . . . 21
5.5 Ablation Study . . . 22

6 Testing and Validation **22**
6.1 Testing Framework and Strategy . . . 22
6.2 Unit Test Cases — Time Series Analyzer . . . 23
6.3 Unit Test Cases — Evaluations . . . 24
6.4 Test Coverage Summary . . . 25

7 GitHub Usage and Version Control **25**
7.1 Repository Information . . . 25
7.2 Branching Strategy . . . 26
7.3 Team Contribution Summary . . . 26
7.4 Commit History Excerpt . . . 26

8 Deployment and Reproducibility **27**
8.1 Environment and Dependencies . . . 27
8.2 Reproduction Steps . . . 27
8.3 Demo Application . . . 28

9 Discussion **28**
9.1 Interpretation of Results . . . 28
9.2 Limitations . . . 28
9.3 Ethical Considerations . . . 29

10 Conclusion and Future Work **29**

10.1 Summary of Contributions . . . 29

10.2 Future Work . . . 30

Appendix A Full Pseudo-code Listing . . . 33

Appendix B Extended Test Case Log . . . 36

Appendix C Individual Contribution Statement . . . 37

List of Figures

1 Exploratory data analysis: category distributions in QA data . . . 9

2 An Overview of System Workflow . . . 13

3 GitHub branch and merge history (exported from GitHub Insights). . . . 26

4 Interactive demo . . . 28

List of Tables

1 Part of the available series from FRED API . . . 8

2 QA data partition sizes . . . 9

3 Model family comparison for this task . . . 11

4 An Overview of Baseline Models . . . 11

5 Agents with progressively increasing capabilities added . . . 12

6 Hyperparameter configuration . . . 15

7 Metrics for Model Evaluation . . . 20

8 Retrieval Evaluation Results . . . 20

9 Summary Evaluation Results . . . 21

10 Ablation Experiment Results on Baseline Model . . . 22

11 Unit test cases for `metrics_computing.py` . . . 23

12 Unit test cases for the evaluations . . . 24

13 Test coverage by module . . . 25

14 Individual GitHub contribution summary . . . 26

1. Introduction

1.1 Problem Statement and Motivation

As a significant breakthrough in Natural Language Processing, Large Language Models (LLMs) are increasingly used for data analysis and decision support across diverse fields and tasks. Despite their strong generative capabilities, LLMs are prone to hallucination, generating plausible yet nonfactual content [Huang et al., 2025]. To address hallucinations, a general-purpose fine-tuning method known as Retrieval Augmented Generation (RAG) has emerged as a promising approach [Lewis et al., 2021]. RAG combines a retriever that identifies relevant documents and a generator that creates coherent responses from this information. By combining LLMs with external knowledge bases, RAG systems can enhance response accuracy and relevance by dynamically incorporating up-to-date and verifiable information into generated outputs, compared to standalone LLMs.

While RAG has been extensively studied in general-knowledge and domain-specific settings, such as healthcare and scientific literature, relatively little work has explored RAG systems grounded in real-time economic data, particularly structured macroeconomic indicators.

To address this gap, this project builds a light-weight Agentic RAG system that bridges natural-language questions and the FRED API. The system is designed to autonomously determine which series to retrieve and over which date range, fetch the data via tool calls, and generate a grounded summary answer.

1.2 Research Questions / Objectives

The core research questions include:

RQ1. Under an autonomous RAG architecture, whether a light-weight open-source LLM (LLaMA3.2) can achieve retrieval and summarization quality comparable to that of a commercial model (GPT-4o-mini).

RQ2. Whether enhanced features such as semantic retrieval, rule-based date parsing, few-shot prompt, and self-checking loops can significantly improve retrieval accuracy and answer quality.

RQ3. Whether fine-tuning based on tool call parameter generation can improve the LLM's tool utilization ability.

RQ4. Whether fine-tuning based on human-generated summaries can improve the quality of LLaMA summarization.

1.3 Report Structure

This report is organized as follows: Section 2 reviews related work; Section 3 describes the data pipeline; Section 4 details model selection and implementation; Section 5 presents results and evaluation; Section 6 covers testing and validation; Section 7 documents version control and team contributions; Section 8 addresses deployment and reproducibility; and Section 10 concludes with future work.

2. Related Work

2.1 Existing Approaches to the Problem

Tool-augmented LLMs [Schick et al., 2023] demonstrate that models can invoke external tools to answer questions beyond their training cutoff. The latest advancements in LLMs have enabled LLM Agents to autonomously handle tasks and reliably interact with external functionalities and APIs. The addition of tool calling, also known as function calling, has allowed complicated user requests to be possible, shining light on new use cases for Agents, such as interacting with a secure domain-specific database, and performing advanced QA tasks over domain-specific knowledge management articles [Lumer et al., 2024]. Agentic RAG transforms the originally static "retrieval-generation" process into a dynamic one by introducing AI agents capable of autonomous decision-making. The agents can plan and perform multi-step reasoning based on task complexity, dynamically select retrieval strategies and invoke external tools, and continuously optimize queries and results through self-reflection. Additionally, it supports multi-agent collaboration to accomplish complex tasks. Compared to traditional RAG, Agentic RAG offers significant enhancements in flexibility, contextual understanding, and the ability to handle complex problems, making it better suited for scenarios requiring real-time information, multi-turn reasoning, and cross-domain decision-making [Singh et al., 2025]. Adaptive RAG offers a balanced strategy that seamlessly adapts between iterative and single-step retrieval-augmented LLMs and non-retrieval methods in response to a range of query complexities [Jeong et al., 2024].

2.2 Relevant Model Architectures

The two baseline LLMs used in this project are LLaMA3.2-3b and GPT-4o-mini. The LLaMA model is deployed locally using Ollama in Docker [Ollama, 2025]. The GPT model is invoked via the API [OpenAI, 2023].

2.3 Datasets and Benchmarks

The database that this project uses is called the Federal Reserve's economic database (FRED), an online database consisting of 100,000+ economic time series data from scores of national, international, public, and private sources [FRED, 2026].

For benchmarks used in this project, three metrics are defined for the retrieval evaluation and three metrics are defined for the summary evaluation. The details will be introduced in section 5. However, it should be noted that most of the metrics we use in this paper are F1 scores because we want to consider the tradeoff of precision and recall in both retrieval and summarization use cases. BERTSCORE is one of the metrics that will be used as a benchmark in this paper, which is a popular automatic evaluation metric for text generation. It computes a similarity score for each token in the candidate sentence with each token in the reference sentence. The key innovative point of this metric is that it takes in contextual embeddings instead of exact matches, which makes the metric correlate better with human judgments and provides stronger model selection performance than existing metrics [Zhang et al., 2019].

2.4 Positioning of This Work

In this paper, we extend the tool-augmented RAG paradigm with a multi-layer validation pipeline to improve response reliability in the macroeconomic domain, where numerical precision and domain-specific insight are critical. The primary goal of this project is to build an agentic RAG system grounded in real-time economic FRED data, capable of accurately retrieving relevant indicators and providing reliable economic trend analysis given a user's natural language query. Also, it should be able to provide an interpretable analysis of macroeconomic indicators for users with limited financial knowledge.

It should be noted that there are currently no projects that are the same as ours, so there is no single case that can be used for direct comparison. However, since there are already many examples of agentic RAGs, our project will draw on existing mainstream designs and adapt them to suit our specific use cases.

3. Data Pipeline and Preprocessing

3.1 FRED Data Repository Description

The main macroeconomic data we use is retrieved from the FRED API. It provides hundreds of curated US macroeconomic indicators. For part of the series data provided by the API, please refer to Table 1. Since users need to know the specific series ID when retrieving data via the API, we

choose 92 commonly used series and create a mapping between series and their specific series IDs.

Table 1: Part of the available series from FRED API

Category	Example Series ID	Frequency
GDP & Growth	GDP, GNP, GDPC1	Quarterly
Labor Market	UNRATE, PAYEMS, CIVPART	Monthly
Inflation	CPIAUCSL, PCEPI, CPILFESL	Monthly
Monetary Policy	FEDFUNDS, DFF, DISCOUNT	Daily / Monthly
Trade	BOPGSTB, EXPGS, IMPGS	Quarterly
Housing	PERMIT, HSN1F, MORTGAGE30US	Monthly / Weekly
Financial Markets	VIXCLS, T10Y2Y, BAA10Y	Daily
Money Supply	M1SL, M2SL, M1V, M2V	Monthly / Weekly

3.2 Fine-tuning/Test Datasets

For summary fine-tuning and retrieval accuracy testing, we use a Question-Answering (QA) dataset that contains hand-curated economic questions annotated with expected series IDs, expected date ranges, and key fact values. Question types include: 1. Single series data query or trend summarization (e.g., "Show GDP for the first quarter of 2024."); 2. Multiple series data query or trend summarization (e.g., "Compare the unemployment rate and inflation rate for 2023."); 3. Questions not related to FRED or the query tool (e.g., "What does GDP stand for?"). Queries also involve different expression styles, such as colloquial and formal, ambiguous and clear, designed to simulate real-world user queries. Each entry in this dataset includes: question text, expected series IDs (list), expected date range (absolute or relative), tool-call-required flag, reference answer, and optionally key facts for data-point-level evaluation. The fine-tuning/test data split and relevant information are given in Table 2.

Note that, to reduce costs, the reference summaries in the dataset used for this project were generated by GPT-4 and subsequently reviewed and edited by human reviewers. For more accurate BERT scores, it is recommended to use manually generated reference summaries. The data was generated in March 2026. Since some of the data is time-sensitive, some benchmarks and values in the dataset may be out-of-date soon.

Table 2: QA data partition sizes

Split	Samples	Notes
Fine-tuning	[727]	[entire dialogue process, from system prompts to issues, tool calls, and the final summary]
Test	[50]	[correct series id and time range for tool calls, reference summaries, key facts; never seen during fine-tuning]

The QA dataset we create includes questions on more than 12 categories of data. The distribution of categories is shown in Figure 1. Most of the questions in the dataset are common data queries; the others include a small number of edge cases, such as out-of-scope questions, questions that do not require data queries, and questions that require further clarification from the user.

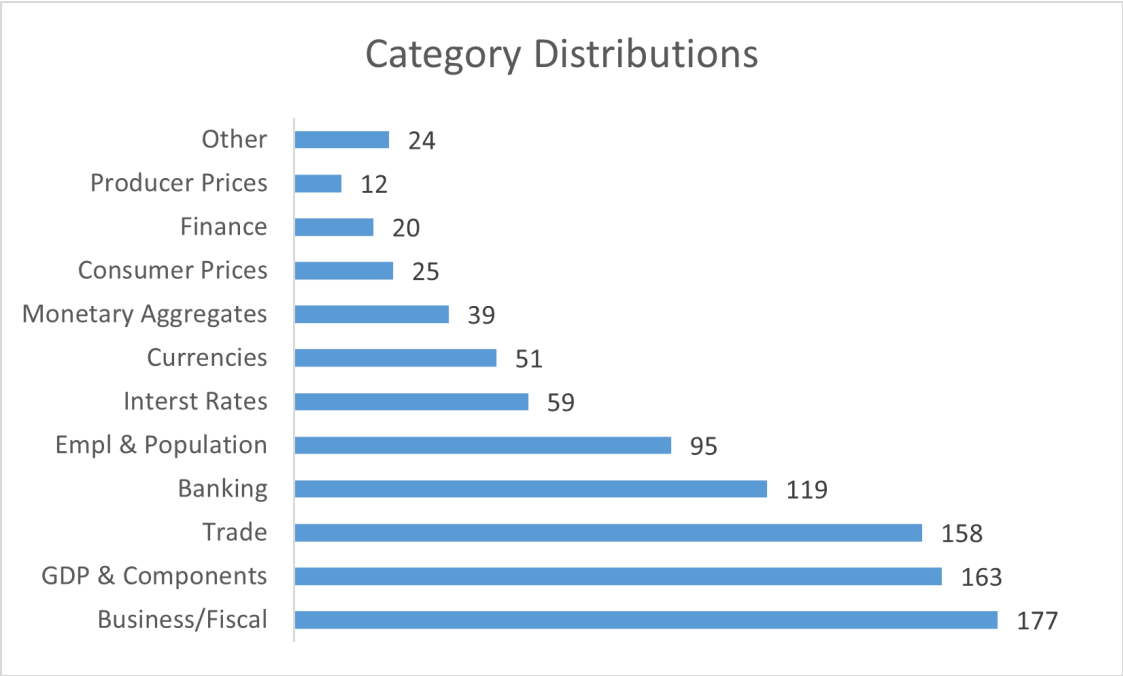


Figure 1: Exploratory data analysis: category distributions in QA data

3.3 Time Series Data Analysis

We create a pipeline to compute common time series data metrics, and apply trend analysis on the data returned by the FRED API. The pipeline will then return analysed results to LLM in JSON format. This approach aims to save tokens and reduce hallucination in large language models by providing fact-based numerical analysis. The pseudocode for the time series analyzer is given below.

This analyzer consists of six core components: data parsing and cleaning, basic statistics, overall change, linear regression trend analysis, rolling volatility comparison, and identification of significant MoM/YoY periods, as well as two output modes: compact and detailed.

3.4 Series Index Guide Generation

Compact Text Series Index Guide

We create a list of the most commonly used series with their corresponding identities, names, update frequency, unit, and category. For each series, we write a detailed semantic description of its definition and economic impact in reality. This guide is used for baseline model system prompts.

Semantic Series Index Guide

Based on the compact text guide, we generate FAISS indices with metadata by encoding the series descriptions into dense embeddings using a sentence-transformer model. These embeddings would be used for dynamic series lookup to format results into LLM-readable context, thereby theoretically improving LLMs' ability to identify the target data associated with user queries and, consequently, their retrieval accuracy. The pseudocode for series indices generation based on descriptions is given below.

Pseudo-code: FAISS Series Index Construction

Require: Series metadata $\mathcal{M}$, pre-trained encoder $\mathcal{E}$ (`all-MiniLM-L6-v2`)

Ensure: FAISS index file `series_index.faiss` and metadata store `series_index_meta.pkl`

Load $\mathcal{M} = \{m_1, m_2, \ldots, m_K\}$ from JSON file, where each m_k contains fields SERIES, INDICATOR, CATEGORY, SUB-CATEGORY, UNITS, PERIOD, DESCRIPTION

// Text serialisation

for each series record $m_k \in \mathcal{M}$ **do**

  Map PERIOD code to human-readable frequency (e.g. `M` $\rightarrow$ `Monthly`)

  Concatenate fields into plain text t_k

end for

// Dense embedding

Load sentence encoder $\mathcal{E}$

$E \leftarrow \mathcal{E}(\{t_1, \ldots, t_K\})$, $E \in \mathbb{R}^{K \times d}$

ℓ_2-normalise each row: $\hat{e}_k \leftarrow e_k / \|e_k\|_2$ (converts inner product to cosine similarity)

// Index construction

Initialise flat inner-product index $\mathcal{I} \leftarrow$ `IndexFlatIP`(d)

Add all normalised embeddings: $\mathcal{I}$.add$(\hat{E})$

// Persistence

Serialise $\mathcal{I}$ to `series_index.faiss`
Serialise $\{\mathcal{M},\ \{t_k\}\}$ to `series_index_meta.pkl`
return $\mathcal{I}$, metadata store

4. Model Selection and Implementation

4.1 Model Family Justification

Table 3: Model family comparison for this task

Family	Strengths	Limitations	Selected?
Classical ML	Interpretable, fast, low data requirements	Limited on complex patterns	✓ / —
Deep Learning	Captures complex patterns, end-to-end	Needs large data, expensive	✓ / —
LLM / Foundation	Strong generalization, few-shot capable	Inference cost, hallucination	✓ / ✓

We use two baseline models: LLaMA 3.2 (3 billion parameters) and GPT-4o-mini as the agent in our system for comparison.

Table 4: An Overview of Baseline Models

Model	Rationale
LLaMA 3.2 (3B)	Locally deployable via Docker; no API cost; fine-tunable; representative small open-source LLM
GPT-4o-mini	Strong instruction-following and tool-use; serves as pseudo-reference for fine-tuning data and BERTScore evaluation

We develop six agent variants, progressively adding capabilities, to demonstrate how system design affects the performance of the RAG system. We systematically compare the two baseline models across three progressively enhanced agent variants: a baseline using a compact text indicator guide, a semantic retrieval version that dynamically selects the most relevant series via FAISS embeddings, and a final version incorporating a rule-based date parser and a three-layer self-check pipeline (relevance gating, parameter validation, and answer completeness verification). Table 5 gives an overview of the differences between agents. Retrieval accuracy is evaluated across all six variants on

a curated question benchmark covering single- and multi-series queries, relative time expressions, and out-of-scope questions. See section 6 for more details.

Table 5: Agents with progressively increasing capabilities added

ID	Model	Series Retrieval	Key Enhancements
M1	LLaMA 3.2	Full text guide	Baseline
M2	LLaMA 3.2	FAISS semantic top-k	Dynamic context reduction
M3	LLaMA 3.2	FAISS semantic top-k	Dynamic context reduction + Check A/B/C + date parser
M4	GPT-4o-mini	Full text guide	Baseline
M5	GPT-4o-mini	FAISS semantic top-k	Dynamic context reduction
M6	GPT-4o-mini	FAISS semantic top-k	Dynamic context reduction + Check A/B/C + date parser

4.2 Model Architecture

The system follows a three-stage agentic pipeline: (1) Tool Call Extraction: Given a user question, the LLM determines which FRED series to query and over what date range; (2) Tool Call Execution: The system calls the FRED API with the extracted parameters and retrieves time-series data; (3) Summary Generation: The LLM receives the retrieved data and generates a natural-language answer. Between these steps, we design a three-Layer self-check pipeline: (A) Relevance Gate: If the maximum cosine similarity between the question and any FRED series is below 0.3, the system returns an early rejection, preventing hallucinated series IDs for out-of-scope questions; (B) Parameter Validation: Each extracted series id is validated against the known FRED series list. Invalid IDs trigger a targeted correction re-prompt; (C) Answer Completeness: After summary generation, a separate LLM call evaluates whether the answer covers all retrieved series and addresses the original question. Incomplete answers are regenerated with targeted prompts. Figure 2 is an overview of the entire system workflow.

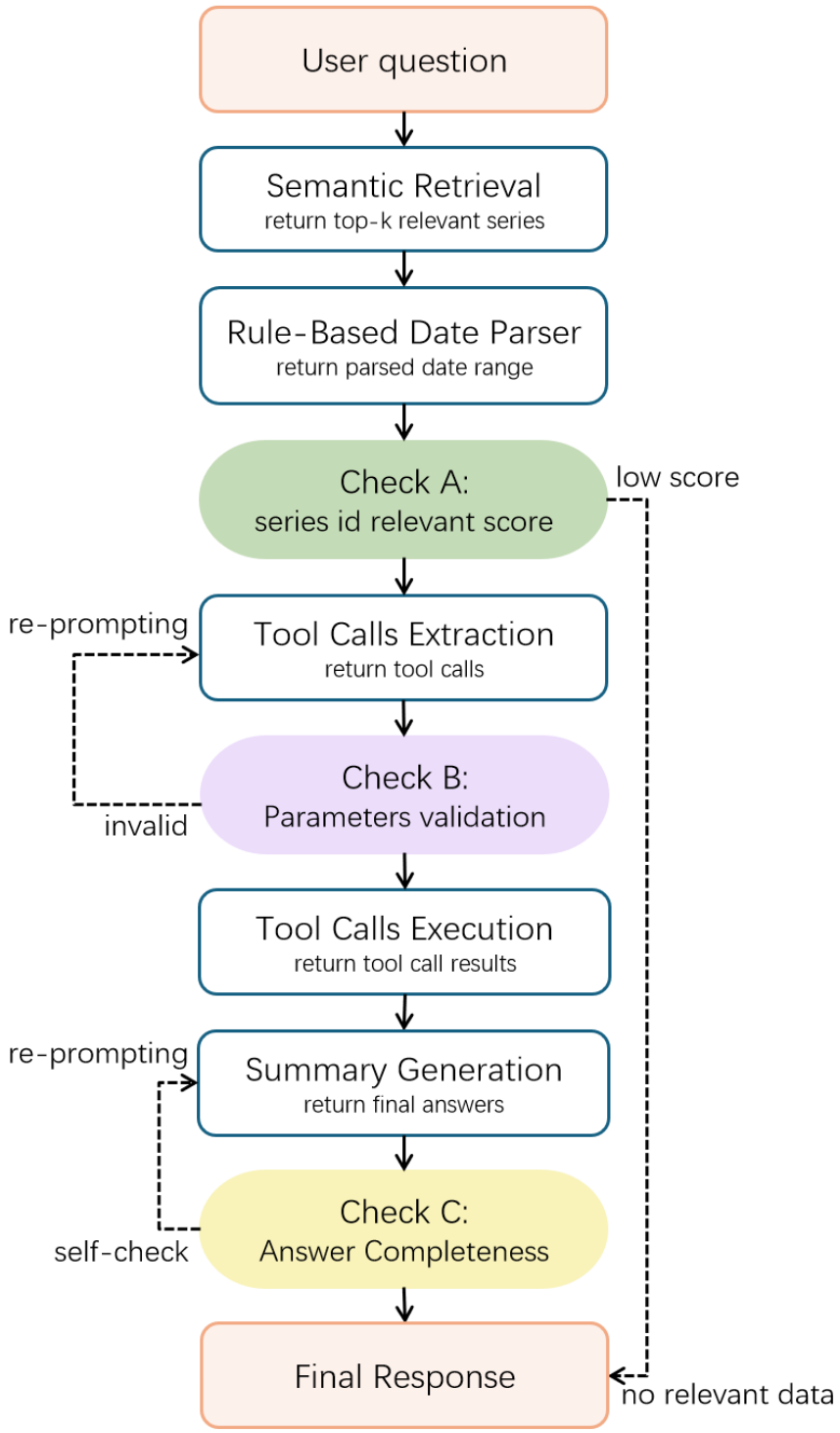


Figure 2: An Overview of System Workflow

4.3 Coding Paradigm and Design Architecture

4.3.1 Paradigm Choice

[Explain whether you used Object-Oriented Programming (OOP), functional programming, or a hybrid, and why that choice is appropriate for your project.]

The software engineering paradigm for this project used a hybrid of OOP, procedural, and functional programming, due to the diverse requirements for different parts of this project.

4.3.2 Module Structure

```
RAG-project/
|-- data/
|   |-- QA.json
|   |-- QA_new.json
|   |-- QA_test.json
|-- files/
|   |-- llama3.2/
|   |-- gpt-4o-mini/
|   |-- human_generated_summary_test.json
|   |-- indicator_guide_compact.txt
|   |-- series_index.faiss
|   |-- series_index_meta.pkl
|-- figs/
|   |-- self_check_flow_chart.png
|-- fine_tune/
|   |-- llama_finetune.ipynb
|-- preparation/
|   |-- build_series_index.py
|   |-- generate_series_description.py
|   |-- indicator_formatter.py
|   |-- QA_gpt_transformer.ipynb
|   |-- wikitable_crawler.py
|-- src/
|   |-- date_parser.py
|   |-- few_shot_examples.py
|   |-- fred_api.py
|   |-- fred_key.py  # should be manually added
|   |-- gpt_api.py
|   |-- gpt_api_final.py
|   |-- gpt_api_semantic_retriever.py
|   |-- gpt_key.py  # should be manually added
|   |-- llama_api.py
|   |-- llama_api_final.py
|   |-- llama_api_semantic_retriever.py
```

```
|    |-- metrics_computing.py
|    |-- output.json
|    |-- series_retriever.py
|-- tests/
|    |-- retrieval_accuracy_benchmark.ipynb
|    |-- retrieval_accuracy_test.py
|    |-- summary_evaluation_benchmark.ipynb
|    |-- summary_evaluation.py
|-- requirements.txt
|-- user_interface.py
|-- README.md
|-- report.pdf
```

Listing 1: Repository directory structure

4.4 Hyperparameters

Table 6 gives some of the hyperparameters we use in our system and during the fine-tuning.

Table 6: Hyperparameter configuration

Component	Parameter	Value
Semantic retriever	`top_k` (series returned)	k=5; tested 3–10
Check A	`min_similarity` threshold	0.3
Date range comparison	`tolerance_days`	30 days
Fine-tune learning rate	`learning_rate`	2e-4
Fine-tune epochs	`num_train_epochs`	3
Fine-tune optimizer	`optim`	adamw_8bit

4.5 Key Components

4.5.1 Prompt Engineering

Semantic Retrieval vs. Full Text Guide

The baseline appends all series descriptions to LLM's system prompt, wasting context tokens and introducing noise. The semantic retriever uses cosine similarity in embedding space to select only the top-k most relevant series for a given question. And only include these most relevant series in the LLM's prompt for tool calls parameters generation, reducing irrelevant options and improving series selection precision.

Few-shot prompting

Few-shot prompting refers to the practice of including several high-quality summary examples in the prompt when asking the model to generate a final summary. To improve the summary quality, we implement a baseline variant with few-shot prompting, and compare the summary quality between baseline LLaMA3.2 and variant LLaMA3.2 with few-shot prompting.

4.5.2 Rule-Based Date Parser

LLMs frequently produce date errors: future end dates, same start/end dates. A deterministic rule-based parser handles the following unambiguous patterns before the LLM is invoked:

1. Quarter expressions: ”Q1 2024”, ”Q3 and Q4 2022” (multi-quarter merge to continuous range)
2. Half-year: ”first half of 2022”, ”second half of 2020”
3. Relative ranges with English number words: ”past two years”, ”last 18 months”, ”past decade”
4. ”Since YEAR” and ”from YEAR to YEAR” range patterns
5. Single unambiguous year: ”in 2023”, ”during 2019”

Ambiguous expressions (e.g., ”recently”, ”lately”, ”before Covid”) are explicitly left unresolved for the LLM, following a conservative design principle that only returns a date range when fully certain.

4.5.3 Three-step Self-check

To verify whether the quality of the results at each stage meets expectations, so that the system can go smoothly to the next step, we design a three-step self-check pipeline. If the results do not meet expectations, the system will perform the appropriate fallback operations.

- Check A: Relevance Gate
- Check B: Parameter Validation
- Check C: Completeness Check

The pseudocodes for the three self-checks are given below.

Pseudo-code: Check A: Relevance Gating

Require: User question q, semantic retriever $\mathcal{R}$, threshold $\tau = 0.3$, Candidate count k

Ensure: Proceed to tool call generation, or early rejection

Retrieve top-k candidate series: $S \leftarrow \mathcal{R}(q,\ k)$, where each $s_i \in S$ carries a cosine similarity score
$s_{\max} \leftarrow \max_{s_i \in S}\ \text{similarity}(s_i)$
if $s_{\max} < \tau$ **then**
    return rejection: *"No relevant FRED series found. This question may be outside FRED's coverage."*
else
    Pass S as the candidate series set to the next stage
end if
return candidate set S

Pseudo-code: Check B: Parameter Validation

Require: Raw tool calls $T = \{(id_i,\ \text{start}_i,\ \text{end}_i)\}$ from LLM, valid series set V, pre-parsed dates $(\hat{s},\ \hat{e})$ from rule-based date parser, max retries R

Ensure: All tool calls carry valid series IDs and well-formed date ranges

// Date resolution (per tool call)
for each tool call $t_i \in T$ **do**
    Priority 1: if $\hat{s}$ and $\hat{e}$ are available, set $(\text{start}_i,\ \text{end}_i) \leftarrow (\hat{s},\ \hat{e})$
    Priority 2: else if LLM-provided dates are absolute `YYYY-MM-DD`, use them directly
    Priority 3: else resolve relative offsets (e.g. `-2y`, `-6m`) to absolute dates
    Priority 4: sanitise via FIXDATEPARAMETERS: cap future end dates to today; expand same-day or inverted ranges to a 1-year window
end for
// Series ID validation
depth $\leftarrow 0$
while $\exists\, t_i \in T$ s.t. $id_i \notin V$ **and** depth $< R$ **do**
    $T_{\text{inv}} \leftarrow \{t_i \in T \mid id_i \notin V\}$
    Append correction hint to q: *"Series $\{id \mid t \in T_{inv}\}$ do not exist. Please use only series from the provided list."*
    $T \leftarrow \text{EXTRACTTOOLCALLS}(q)$
    depth $\leftarrow$ depth $+ 1$
end while
return validated tool call list T

Pseudo-code: Check C: Answer Completeness Verification

Require: Question q, draft answer a, successful API results R, max self-check loops L
Ensure: Final answer covers all retrieved series and fully addresses q

```
// Guard: skip for simple queries
if |R| ≤ 2 then
    return a
end if
loop ← 0
while loop < L do
    check ← LLM-VERIFY(q, a, {r.series_id | r ∈ R})   (returns JSON with complete, missing_series, question_addressed, gap)
    if check.complete and check.question_addressed then
        break
    end if
    hint ← ∅
    if not check.complete then
        hint ← hint + "Missing series: " + check.missing_series
    end if
    if not check.question_addressed then
        hint ← hint + check.gap
    end if
    a ← LLM-REGENERATE(q, hint)
    loop ← loop + 1
end while
return a
```

4.6 Fine-tuning

If using an LLM: describe your prompting strategy (zero-shot, few-shot, chain-of-thought), system prompt design, and any fine-tuning (LoRA, QLoRA, full fine-tuning). If using a pre-trained DL model: describe transfer learning strategy, frozen vs. fine-tuned layers.

LLaMA 3.2 is further fine-tuned on the GPT-generated reference summaries to close the summarization quality gap. See Section 5 for more details.

- Base model: LLaMA 3.2-3B
- Fine-tuning method: LoRA [Hu et al., 2021], QLoRA [Dettmers et al., 2023] for parameter-efficient training

- Training data: QA_new.json (question, FRED data, reference summary) triples
- Objective: Supervised next-token prediction on reference summaries conditioned on question + retrieved data
- Reference: Unsloth [Daniel Han and team, 2023]

4.7 Why Compare Multiple Models and Variants

- LLaMA 3.2 vs GPT-4o-mini: Assess whether a locally-deployable open-source model can close the performance gap with a commercial API for cost, privacy, and latency considerations.
- Semantic Retriever vs Baseline: Quantify retrieval accuracy improvement from dynamic series selection vs. full-context enumeration.
- Semantic Retriever vs Final: Isolate the contribution of the date parser and self-check pipeline to retrieval accuracy and summary quality.
- Fine-tuned LLaMA vs Baseline: Determine whether supervised fine-tuning on GPT-generated summaries improves LLaMA summarization quality.
- Few-shot LLaMA vs Baseline: Determine whether few-shot prompting improves LLaMA summarization quality.

5. Results and Model Evaluation

For retrieval evaluation, we did 6 experiments: 3 versions of 2 models. For summary evaluation, we did 12 experiments: 3 versions of 3 models and 3 versions of the baseline model with few-shot prompting.

5.1 Evaluation Metrics

To evaluate the system, we defined six metrics across two dimensions: retrieval quality and summary quality. Table 7 shows details of the metrics.

5.2 Retrieval Accuracy Evaluation

For the retrieval evaluation, adding the semantic retriever brings LLaMA's series id score up around 20%. Date parser also improves LLaMA's date range score for around 10%. Combining all three features in the final variant improves both the series id score and date range score. For GPT variants,

Table 7: Metrics for Model Evaluation

Metric	Scope	Description
Series id F1 Score	Retrieval	F1 score between retrieved ids and correct ones
Date Range Score	Retrieval	Continuous score (0-1) based on start/end date proximity
Overall Retrieval	Retrieval	average of Series F1 and Date Range Score
BERTScore	Summary	Semantic similarity between LLM's summary and reference
Hallucination Rate	Summary	Numeric claims in LLM's summary vs. raw output from API
Key Fact Coverage	Summary	Whether annotated key data points appear in the answer

the three new components did not lead to as significant an improvement in scores as Llama did, but the result still demonstrates that these components play a role in enhancing system performance. Table 8 shows the detailed retrieval evaluation results.

Table 8: Retrieval Evaluation Results

Agent	Series ID Score	Date Range Score	Overall Score
M1	0.75	0.82	0.79
M2	0.91	0.84	0.87
M3	0.94	0.94	0.94
M4	0.97	0.99	0.98
M5	0.99	0.98	0.98
M6	0.97	0.99	0.98

5.3 Summary Quality Evaluation

For the summary evaluation, GPT again outperforms LLaMA across all metrics, but the gap narrows considerably with the enhancements. The full enhancement pipeline brings LLaMA's hallucination rate down to 30.7% while maintaining key fact coverage at 87.8%. GPT with all enhancements achieves the best overall performance: a key fact coverage of 96.7% and a hallucination rate of just 20.9%, confirming that self-check mechanisms are effective for quality control. One tradeoff worth noting: GPT generates significantly longer summaries — around 1,200 words — compared to LLaMA's 800 words. Depending on the use case, this may or may not be desirable.

Regarding fine-tuning, the fine-tuned LLaMA without any enhancements underperforms the base LLaMA. This is likely because fine-tuning on summaries alone led to catastrophic forgetting and overfitting on the linguistic style instead of the reasoning logic of the training data. When combined with all pipeline enhancements, the fine-tuned model recovered and achieved performance slightly better than that of the non-fine-tuned enhanced version.

It is also surprising to see that the few-shot prompting approach for LLaMA achieves 90.9% of GPT's coverage at zero API cost, making it a recommended option for budget-conscious deployments, without the cost of fine-tuning.

In summary, the combination of the three mechanisms proves effective for both models. Table 9 shows the detailed retrieval evaluation results.

Table 9: Summary Evaluation Results

Agent	Key Fact Coverage	Hallucination Rate	Average Length	BertScore
LLaMA	0.8750	0.4346	719.1600	0.8566
LLaMA (FAISS)	0.8650	0.2516	710.6800	0.8620
LLaMA (final)	0.8781	0.3074	883.4800	0.8609
GPT	0.9304	0.2467	1258.96	0.8632
GPT (FAISS)	0.9667	0.2225	1231.7000	0.8656
GPT (final)	0.9667	0.2090	1246.1800	0.8784
LLaMA (finetuned)	0.8100	0.3898	734.6800	0.8447
LLaMA (FAISS, finetuned)	0.8824	0.3553	869.4200	0.8529
LLaMA (final, finetuned)	0.8923	0.3074	863.2600	0.8568
LLaMA (few shot)	0.8377	0.4172	695.3200	0.8557
LLaMA (FAISS, few shot)	0.8856	0.3156	744.2000	0.8626
LLaMA (final, few shot)	0.9088	0.2598	600.2800	0.8674

5.4 Error Analysis

For retrieval failure, a common scenario is that, for vague queries, due to the large number of series in the database, there are many series with similar functions that are difficult for an LLM to distinguish and choose. For example, regarding unemployment, there are Civilian Total Unemployment Rate, Long Term Unemployment: $>$ 27 WKS, and Length of Unemployment. When a user queries about the unemployment situation, it would be hard for LLM to decide which single one or multiple ones to retrieve. For this case specifically, there may be many different retrieval strategies; however, due to limitations in the evaluation data and the code, our benchmark classifies some acceptable strategies as incorrect retrievals.

For the summary quality situation, the main issues are forgetting and hallucinations in LLMs. Even when provided with data and statistical results from functional programming calculations, and the prompt is appropriately shortened, LLMs still exhibit these issues. LLMs tend to rephrase numbers—for example, by changing units—which can cause our code to misinterpret the number as a product of the model's hallucination.

5.5 Ablation Study

To better understand the contribution of each component, I ran an ablation experiment on the LLaMA baseline. Starting from the baseline retrieval score of 0.785, both the retriever and the date parser improve retrieval performance. The self-checks alone don't change retrieval at all, as expected, but the full enhanced system achieves a +19.75% retrieval improvement, showing a cumulative effect. On the summarization side, the self-checks contribute the most. The full pipeline achieves a +10.04% cumulative improvement over the baseline.

Note that we used 1 to represent the baseline summarization performance. This is because we consider that simply summing up the values of the three metrics is not able to give a meaningful value. The formula we used to calculate the percentage change for summarization performance is shown below the table, so that it makes it easier to see the impact of the enhancement on the baseline model's performance.

In general, each component has a distinct and positive effect. Semantic retrieval matters most for retrieval quality, and self-check mechanisms matter most for summary quality. And the cumulative effect is considerable. Table 10 shows the detailed ablation study results.

Table 10: Ablation Experiment Results on Baseline Model

Component	Retrieval Performance	Summarization Performance
Baseline	0.785	1.000
+ Semantic Retriever	0.910 (+15.92%)	1.066 (+6.60%)
+ Date Parser	0.893 (+13.71%)	1.018 (+1.81%)
+ Checks	0.785 (+0.00%)	1.090 (+8.97%)
Full Enhanced	0.940 (+19.75%)	1.104 (+10.04%)

Note: percentage change for summarization performance $=$
$\frac{1}{3}\left(\Delta\%\textit{Key Fact Coverage} - \Delta\%\textit{Hallucination} + \Delta\%\textit{BertScore}\right)$

6. Testing and Validation

6.1 Testing Framework and Strategy

We used the unittest library for completing unit tests. We tested evaluation methods and the Time Series Analyzer because these methods require consideration of various possibilities and precise calculations. We did not test the agent's main process file because tests involving API calls are relatively complex.

```
# unit_tests/unit_tests.py
class TestExtractNumbers(unittest.TestCase):
    def test_numbers_match(self):
```

```python
        """
        Test `_numbers_match()` which compares numeric claims from LLM's response with raw api outputs.
        """
        sample1, sample2 = 27.37, 27.370000
        self.assertEqual(sample1, sample2)
        sample3, sample4 = 27.37, 27.3699999
        self.assertEqual(sample3, sample4)
        sample5, sample6 = 0.0000001, 0.00001
        self.assertEqual(sample5, sample6)

    def test_extract_floats_recursive(self):
        """
        Test `_extract_floats_recursive()` which extracts numeric claims from fred_api outputs.
        """
        sample = [
            {
                "success": True,
                "series_id": "FEDFUNDS",
                "analysis": {
                    "overview": {"data_points": 12},
                    "key_statistics": {"average": 4.1, "std_deviation": 0.29},
                    "trend_analysis": {"total_change": {"absolute": -0.69, "percentage": -15.94}},
                }
            }
        ]
        result = _extract_floats_recursive(json.loads(sample))
        self.assertEqual(result, [12.0, 4.1, 0.29, -0.69, -15.94])
```

Listing 2: Example pytest unit test for the preprocessing module

6.2 Unit Test Cases — Time Series Analyzer

Table 11: Unit test cases for `metrics_computing.py`

ID	Description	Input	Expected Output	Status
UT-01	Empty input raises `ValueError`	Empty list `[]`	`ValueError` raised	**PASS**
UT-02	Non-numeric values dropped	6-row list with one `"bad"` string value	DataFrame has 5 rows; no NaN remains	**PASS**

continued on next page

ID	Description	Input	Expected Output	Status
UT-03	Basic statistics on known series	$[10, 20, 30, 40, 50, 60]$	max=60, min=10, mean=35, latest=60, earliest=10	**PASS**
UT-04	Trend detection on increasing series	$[10, 20, 30, 40, 50, 60]$	`trend="increasing"`, `strength="strong"`	**PASS**
UT-05	Trend detection on constant series	$[100, 100, \ldots]$	`trend="stable"`	**PASS**
UT-06	Volatility returns `None` when series too short	Series of length 3, default window$=6$	`None`	**PASS**
UT-07	Compact summary contains correct keys only	$[10, 20, 30, 40, 50, 60]$, `compact_mode=True`	6 expected keys present, `full_timeseries` absent	**PASS**

6.3 Unit Test Cases — Evaluations

Table 12: Unit test cases for the evaluations

ID	Description	Input	Expected Output	Status
MT-01	Number extraction from LLM response	Free-text answer containing wages, percentages, and hours	Deduplicated float list in text order; years and month-day tokens excluded	**PASS**
MT-02	Numeric match — identical values	27.37 vs 27.370000	`True`	**PASS**
MT-03	Numeric match — within relative tolerance	27.37 vs 27.3699999	`True`	**PASS**
MT-04	Numeric match — near-zero absolute tolerance	0.0 vs 0.05, `abs_tol=0.1`	`True`	**PASS**
MT-05	Numeric match — clearly different values	27.37 vs 30.0	`False`	**PASS**
MT-06	Recursive float extraction from nested dict	Nested API result dict with `success=True`, booleans, and numeric fields	`[12.0, 4.1, 0.29, -0.69, -15.94]`, booleans excluded	**PASS**

ID	Description	Input	Expected Output	Status
MT-07	Series ID perfect match (F1)	Predicted = {GDP, UNRATE}, expected = {GDP, UNRATE}	F_1=1.0, missing = ∅, extra = ∅	**PASS**
MT-08	Series ID with missing and extra (F1)	Predicted = {GDP, CPIAUCSL}, expected = {GDP, UNRATE}	F_1=0.5, missing ={UNRATE}, extra={CPIAUCSL}	**PASS**
MT-09	Series ID empty prediction (F1)	Predicted = ∅, expected = {GDP}	F_1 = 0.0	**PASS**
MT-10	Date range — no expected range	No expected date range	score = 1.0	**PASS**
MT-11	Date comparison — within tolerance	date1 = 2024-01-01, date2 = 2024-01-20, tol = 30d	score = 1.0	**PASS**
MT-12	Date comparison — actual end before expected end	date1 = 2024-12-31, date2 = 2024-06-01	score= 0.0	**PASS**

6.4 Test Coverage Summary

Table 13: Test coverage by module

Module	Files	Covered	Coverage %
`metrics_computing.py`	[1]	[1]	[100%]
`model_api_version.py`	[8]	[0]	[0%]
`LLaMA_finetune.py`	[1]	[0]	[0%]
`evaluation.py`	[2]	[2]	[100%]
`Other supporting documents`	[2]	[0]	[0%]
Total	[14]	[3]	**[21%]**

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** https://github.com/dcr225619/DS5500-RAG
- **Visibility:** Public

- **Primary Branch:** `master`
- **Total Commits:** 41 (as of April 15, 2026)

7.2 Branching Strategy

Since there's only one member in this team, and no feature branch was designed, all the updates were pushed to `master` via pull requests.

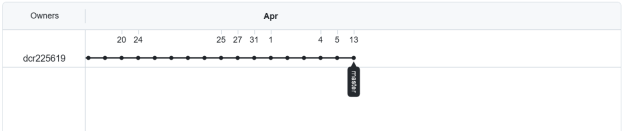


Figure 3: GitHub branch and merge history (exported from GitHub Insights).

7.3 Team Contribution Summary

Table 14: Individual GitHub contribution summary

Member	Primary Responsibilities	Commits	PRs Opened	PRs Reviewed
[Churou Deng]	[Data pipeline, EDA]	[Churou Deng]	[Churou Deng]	[Churou Deng]
[Churou Deng]	[Model architectur]	[Churou Deng]	[Churou Deng]	[Churou Deng]
[Churou Deng]	[Fine-tuning]	[Churou Deng]	[Churou Deng]	[Churou Deng]
[Churou Deng]	[Evaluation]	[Churou Deng]	[Churou Deng]	[Churou Deng]
[Churou Deng]	[Deployment]	[Churou Deng]	[Churou Deng]	[Churou Deng]

7.4 Commit History Excerpt

```
daf416e (HEAD -> master, origin/master) update-apr16
7e3afd2 structural adjustment
d9a1a32 update-apr5
3c77904 update-apr4
db5ac8f update-apr1
f257d6d update-apr1
bf93a98 update-apr1
d4f2f24 update-mar31
f0190de update-mar27
fa55a15 update-mar25
f2c7f64 update-mar24
555a992 update-mar24
38e3209 update-mar24
d600533 update-mar24
00b6088 update-mar24
```

```
46af4c5 update-mar20
8f6435b update-mar18
356a990 update-mar18
70752df update-mar18
24f99ac update-mar18
6f8b7aa update-mar14
563b65b update-mar13
7ecc675 update-mar13
```

Listing 3: Recent commit log (git log –oneline)

8. Deployment and Reproducibility

8.1 Environment and Dependencies

[List key packages and versions. Explain how to recreate the environment.]

```
numpy>=1.24.0,<2.0.0
pandas>=2.0.0,<3.0.0
scipy>=1.10.0,<2.0.0
sentence-transformers>=2.2.0,<3.0.0
bert-score>=0.3.13
faiss-cpu>=1.7.4  # use faiss-gpu if you have CUDA
requests>=2.31.0,<3.0.0
openai>=1.0.0,<2.0.0
joblib>=1.3.0,<2.0.0
python-dateutil>=2.8.0,<3.0.0
streamlit>=1.28.0,<2.0.0
plotly>=5.17.0,<6.0.0
```

Listing 4: requirements.txt excerpt

8.2 Reproduction Steps

Pseudo-code: Reproduction Procedure

Require: see requirements.txt on Github for details

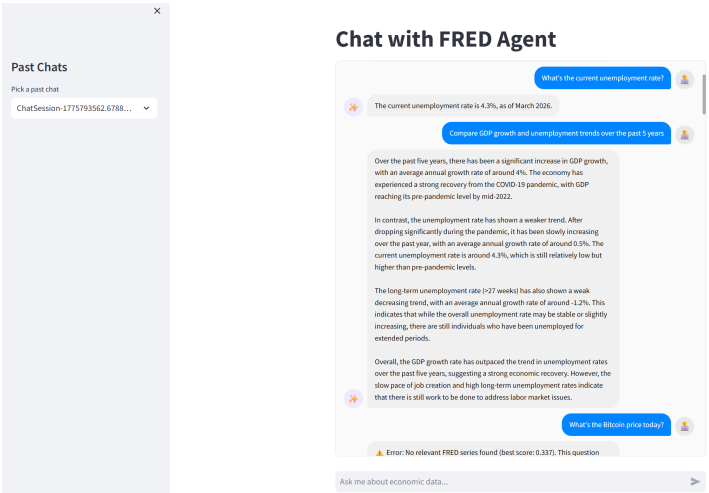
Ensure: Reproducible results matching those reported in Section 5

1: **git clone** https://github.com/dcr225619/DS5500-RAG.git
2: **cd** [repo]
3: **pip install** -r requirements.txt
4: Run any file according to the instructions in readme.md
5: **streamlit run** user_interface.py {Launches interactive demo}

8.3 Demo Application

Run user_interface.py with `streamlit run user_interface.py` to start the application. Start a new chat and enter your FRED data query. You can modify the file user_interface.py to use a different agent.

Figure 4: Interactive demo



9. Discussion

9.1 Interpretation of Results

As mentioned earlier, there are currently no studies that are identical to this project. However, based on the results of other RAG systems, we think there is significant room for improvement in this project. According to our experimental results, we have reason to believe that by using a larger model specialized for the financial sector, we could further enhance the model's performance and stability. On the other hand, given our commitment to using smaller open-source models, we consider that through a series of enhancements, we have achieved competitive results relative to RAG systems in other areas. Future research directions can be divided into two directions: either continuing to optimize small, open-source models or using better models to pursue higher performance and stability.

9.2 Limitations

1. **Data limitations:** Currently, we have only introduced the FRED API to the agent, which limits the model's data sources. We hope to incorporate additional data, such as financial news, in the future to enhance our system's capabilities.

2. **Model limitations:** At present, we are not using models specialized for economics. Therefore, we have designed the model to focus more on analyzing the data itself rather than interpreting the underlying meaning. We hope to improve the model's understanding of economic data in the future, enabling it to generate more in-depth summaries.

9.3 Ethical Considerations

Similar to most LLMs, the agents in this project also face certain ethical considerations. First, the misinformation and hallucination challenge. LLMs can generate false, misleading information (hallucinations) convincingly, which can be misused for malicious purposes. Second, the accountability gaps. It is often unclear who is responsible when AI-driven systems produce harmful or incorrect outputs, in this case, incorrect economic information. It is often difficult to clearly assign responsibility, which may result in financial losses.

Solutions include further enhancing the reliability of the system's output data through a more rigorous review process, as well as reminding users to treat the system's output data and conclusions with caution.

10. Conclusion and Future Work

10.1 Summary of Contributions

In this project, we built an agentic RAG system that makes macroeconomic data from FRED accessible and interpretable to general users without requiring any prior knowledge of API details, or economic expertise. Through three iterative enhancements, the system achieves strong performance. For the Final LLaMA variant, we achieved

1. overall retrieval score of 0.94
2. hallucination rates down to 30%
3. key fact coverage rate to 87%
4. BERTSCORE to 0.86

According to our experiments, we found out that: 1. Splitting task matters: separating retrieval parameter generation from summarization and using the right tool for each improve the performance. 2. Structured pre-computation reduces hallucination: running statistical analysis before the summarization LLM call gives the model grounded facts to work with, rather than asking it to reason from raw numbers. 3. Few-shot prompting is a strong, cost-effective and recommended approach for this task.

10.2 Future Work

1. [News Integration] Right now, the system answers questions purely based on FRED's quantitative time-series data. But users often want to understand why a trend is happening, not just what the trend is. We can integrate real-time news retrieval via NewsAPI, so the system can combine the economic indicators with current economic events, policy announcements.

2. [Multi-agent System] In our current system, one LLM does both the api parameter generation and the summarization jobs. Introducing another agent, so that each agent can be assigned a specific task, may further improve the stability and accuracy of the entire system.

3. [Higher System Reliability] We can further reduce hallucination and enhance the reliability of the system's output data through a more rigorous review process or stronger models.

References

Michael Han Daniel Han and Unsloth team. Unsloth, 2023. URL https://github.com/unslothai/unsloth.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. 05 2023. doi: 10.48550/arxiv.2305.14314. URL https://openreview.net/forum?id=OUIFPHEgJU.

FRED. Getting to know fred, 2026. URL https://fredhelp.stlouisfed.org/#fred-data.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, January 2025. ISSN 1558-2868. doi: 10.1145/3703155. URL http://dx.doi.org/10.1145/3703155.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.389. URL https://aclanthology.org/2024.naacl-long.389/.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL https://arxiv.org/abs/2005.11401.

Elias Lumer, Vamse Kumar Subbiah, James A. Burke, Pradeep Honaganahalli Basavaraju, and Austin Huber. Toolshed: Scale tool-equipped agents with advanced rag-tool fusion and tool knowledge bases, 2024. URL https://arxiv.org/abs/2410.14594.

Ollama, 2025. URL https://hub.docker.com/r/ollama/ollama.

OpenAI. Gpt-4o mini model — openai api, 2023. URL https://developers.openai.com/api/docs/models/gpt-4o-mini.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2501.09136.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *arXiv preprint*, 2019. URL https://openreview.net/forum?id=SkeHuCVFDr.

Appendix A. Full Pseudo-code Listing

A.1 FRED LLM Agent Pipeline

See Section 4 for details.

Pseudo-code: FRED LLM Agent Inference Pipeline

Require: User query q, LLM $\mathcal{M}$, series retriever $\mathcal{R}$, FRED API $\mathcal{F}$, similarity threshold τ, top-k

Ensure: Generated economic analysis a

// Step 1: Tool Call Extraction
Retrieve top-k candidate series: $\mathcal{S}_k \leftarrow \mathcal{R}.\textsc{Retrieve}(q,\ k)$
$\sigma \leftarrow \max_{s \in \mathcal{S}_k} \text{sim}(s,\ q)$
// Check A: out-of-scope detection
if $\sigma < \tau$: **return** "No relevant FRED series found"
Pre-parse date range: $(d_{\text{start}},\ d_{\text{end}}) \leftarrow \textsc{DateParser}(q)$
Build dynamic indicator guide: $g \leftarrow \textsc{BuildGuide}(\mathcal{S}_k)$
Construct system prompt $p_{\text{sys}} \leftarrow g$ and call LLM: $\mathcal{T} \leftarrow \mathcal{M}(p_{\text{sys}},\ q)$
for each tool call $t_i \in \mathcal{T}$ **do**
  t_i.dates $\leftarrow \textsc{ResolveDates}(t_i,\ d_{\text{start}},\ d_{\text{end}})$
end for
// Check B: invalid series re-prompt
Validate series IDs: $\mathcal{T} \leftarrow \textsc{ValidateSeries}(\mathcal{T},\ q)$
// Step 2: Tool Call Execution
$\mathcal{A} \leftarrow \emptyset$
for each validated call $t_i \in \mathcal{T}$ **do**
  $\mathcal{A}_i \leftarrow \mathcal{F}.\textsc{Query}(t_i.\text{series_id},\ t_i.\text{dates})$
  $\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathcal{A}_i\}$
end for
// Step 3: Final Answer Generation
Construct messages $\mathcal{H} \leftarrow [p_{\text{sys}},\ q,\ \mathcal{T},\ \mathcal{A}]$
Generate answer: $a \leftarrow \mathcal{M}(\mathcal{H})$
// Check C: completeness self-check
if $|\mathcal{T}| > 2$: $c \leftarrow \mathcal{M}.\textsc{Validate}(q, a, \mathcal{A})$
  if $\neg\, c$.complete **or** $\neg\, c$.question_addressed: regenerate $a \leftarrow \mathcal{M}(\mathcal{H} + \text{hint})$
return a

A.2 FRED API Pipeline

See Section 3 for details.

Pseudo-code: FRED API Data Retrieval Pipeline

Require: Series ID s, start date d_{start}, end date d_{end}, compact mode flag β

Ensure: Analysis summary Σ or error message

Load indicator metadata: $\mathcal{I} \leftarrow \textsc{LoadMetadata}()$
$(\text{freq}, \text{name}, \text{units}) \leftarrow \mathcal{I}[s]$
// Frequency boundary check
if $(d_{\text{end}} - d_{\text{start}}) < \delta_{\text{freq}}$: adjust d_{start}, d_{end} by $\pm\, \delta_{\text{freq}}/2$
Fetch observations: $\mathcal{O} \leftarrow \text{GET}(\text{FRED}, s, d_{\text{start}}, d_{\text{end}}, \text{freq})$
if request fails: **return** error
Filter missing values: $\mathcal{O} \leftarrow \{o \in \mathcal{O} \mid o.\text{value} \neq \texttt{``.''}\}$
if $\mathcal{O} = \emptyset$: **return** "No data in range"
Initialize analyzer: $\Phi \leftarrow \textsc{TimeSeriesAnalyzer}(\mathcal{O})$
if $|\mathcal{O}| < 5$: $\Sigma \leftarrow \Phi.\textsc{Summarize}(\text{compact} = \top, \text{inflections} = \bot)$
elif β **or** $|\mathcal{O}| < 180$: $\Sigma \leftarrow \Phi.\textsc{Summarize}(\text{compact} = \top, \text{inflections} = \top)$
else: $\Sigma \leftarrow \Phi.\textsc{Summarize}(\text{compact} = \bot, \text{inflections} = \top)$
return $\{s, \text{name}, \text{units}, \Sigma, \mathcal{O}\}$

A.3 Time Series Analyzer Pipeline

See Section 3 for details.

Pseudo-code: Time Series Analyzer

Require: Observations $\mathcal{O} = \{(d_i, v_i)\}$, compact mode β, inflection flag γ

Ensure: Analysis summary Σ

Parse and sort $\mathcal{O}$ into DataFrame $\mathbf{X}$ indexed by date; drop NaN
// Basic Statistics
$\mu \leftarrow \text{mean}(\mathbf{X}), \quad \sigma \leftarrow \text{std}(\mathbf{X}), \quad v_{\max}, v_{\min} \leftarrow \max(\mathbf{X}), \min(\mathbf{X})$
// Trend Assessment via Linear Regression
Fit $\hat{v} = \alpha t + b$ on $\mathbf{X}$; compute $\rho \leftarrow \text{corr}(t, \mathbf{X})$

$$\text{trend} \leftarrow \begin{cases} \text{stable} & |\rho| < 0.3 \\ \text{increasing} & \alpha > 0 \\ \text{decreasing} & \alpha \leq 0 \end{cases}, \quad \text{strength} \leftarrow \begin{cases} \text{weak} & |\rho| < 0.5 \\ \text{moderate} & |\rho| < 0.8 \\ \text{strong} & \text{otherwise} \end{cases}$$

// Volatility Analysis
if $|\mathbf{X}| \geq 12$: compute rolling std $\rightarrow$ compare recent vs. earlier window volatility
// Notable Periods

Compute period-over-period changes; record $\arg\max$ and $\arg\min$ of MoM and YoY $\Delta\%$
// Inflection Point Detection
if γ **and** $|\mathbf{X}| \geq 5$: detect peaks and troughs via $\textsc{FindPeaks}(\mathbf{X},\ \epsilon{=}0.25\sigma)$
// Summary Assembly
if β: $\Sigma \leftarrow \{|\mathbf{X}|,\ \text{range},\ v_{\text{latest}},\ v_{\max},\ v_{\min},\ \text{trend},\ \Delta\%_{\text{total}}\}$
else: $\Sigma \leftarrow \{\text{overview},\ \text{key_stats},\ \text{trend},\ \text{notable_changes},\ \text{volatility},\ \text{inflections}\}$
return Σ

A.4 Retrieval Accuracy Evaluation

See Section 5 for details.

Pseudo-code: Retrieval Accuracy Evaluation

Require: Test suite $\mathcal{Q} = \{(q_j, \mathcal{E}_j, \mathcal{D}_j, \text{flag}_j)\}$, agent $\mathcal{M}$
Ensure: Per-case scores and aggregate summary

for each test case $(q_j, \mathcal{E}_j, \mathcal{D}_j, \text{flag}_j) \in \mathcal{Q}$ **do**
  Extract tool calls: $\mathcal{T}_j \leftarrow \mathcal{M}.\textsc{ExtractToolCalls}(q_j)$
  if $\text{flag}_j = \perp$ *// non-tool-call question*
    if $\mathcal{T}_j \neq \emptyset$: score $\leftarrow 0$ **else**: score $\leftarrow 1$ **continue**
  if extraction failed: score $\leftarrow 0$ **continue**
  // Series ID Evaluation (F1)
  $\mathcal{P} \leftarrow \mathcal{T}_j.\text{series_ids}, \quad \mathcal{E} \leftarrow \mathcal{E}_j$
  precision $\leftarrow |\mathcal{P} \cap \mathcal{E}| \,/\, |\mathcal{P}|, \quad$ recall $\leftarrow |\mathcal{P} \cap \mathcal{E}| \,/\, |\mathcal{E}|$
  $F_1 \leftarrow 2 \cdot \text{precision} \cdot \text{recall} \,/\, (\text{precision} + \text{recall})$
  // Date Range Evaluation
  for each call $t_i \in \mathcal{T}_j$ **do**
    Resolve relative dates in $\mathcal{D}_j$ to absolute YYYY-MM-DD
    $s_{\text{start}} \leftarrow \textsc{CompareDate}(t_i.d_{\text{start}},\ \mathcal{D}_j.d_{\text{start}},\ \epsilon{=}30\text{d})$
    $s_{\text{end}} \leftarrow \textsc{CompareDate}(\mathcal{D}_j.d_{\text{end}},\ t_i.d_{\text{end}},\ \epsilon{=}30\text{d})$
    $s_{\text{date},i} \leftarrow (s_{\text{start}} + s_{\text{end}}) \,/\, 2$
  end for
  $s_{\text{date}} \leftarrow \text{mean}(\{s_{\text{date},i}\})$
  $\text{score}_j \leftarrow 0.5 \cdot F_1 + 0.5 \cdot s_{\text{date}}$
end for
$\bar{F}_1 \leftarrow \text{mean}(\{F_1^j\}), \quad \bar{s}_{\text{date}} \leftarrow \text{mean}(\{s^j_{\text{date}}\}), \quad \bar{s} \leftarrow \text{mean}(\{\text{score}_j\})$
return per-case results and $\{\bar{F}_1,\ \bar{s}_{\text{date}},\ \bar{s}\}$

A.5 Summary Factuality Evaluation

See Section 5 for details.

Pseudo-code: Summary Factuality Evaluation

Require: Model answer a, API results $\mathcal{A}$, key facts $\mathcal{K}$ (optional), tolerances δ_r, δ_a

Ensure: Key-fact coverage rate r_{kf} and hallucination rate r_{hal}

// Ground Truth Construction
$\mathcal{G} \leftarrow \emptyset$
for each result $\mathcal{A}_i \in \mathcal{A}$ **do**
  Recursively extract all numeric values from $\mathcal{A}_i$.analysis and $\mathcal{A}_i$.data
  $\mathcal{G}[\mathcal{A}_i.\text{series_id}] \leftarrow$ deduplicated float set
end for
// Part 1: Key Fact Coverage
if $\mathcal{K} = \emptyset$: $r_{\text{kf}} \leftarrow 1.0$ **else**:
  $\mathcal{N}_a \leftarrow \textsc{ExtractNumbers}(a)$ *// strip commas, skip years and month-day tokens*
  covered $\leftarrow 0$
for each $(s, d, v^*) \in \mathcal{K}$ **do**
  if $\exists\, n \in \mathcal{N}_a$ s.t. $|n - v^*| \,/\, |v^*| \leq \delta_r$ **or** $|n - v^*| \leq \delta_a$: covered $+= 1$
end for
  $r_{\text{kf}} \leftarrow$ covered $/\, |\mathcal{K}|$
// Part 2: Numeric Faithfulness (Hallucination Detection)
$\mathcal{N}_a \leftarrow \{n \in \textsc{ExtractNumbers}(a) \mid |n| \geq 1.0\}$
$\mathcal{G}_{\text{all}} \leftarrow \bigcup_s \mathcal{G}[s]$
ungrounded $\leftarrow \{n \in \mathcal{N}_a \mid \nexists\, g \in \mathcal{G}_{\text{all}} \text{ s.t. } n \approx_{\delta_r, \delta_a} g\}$
$r_{\text{hal}} \leftarrow |\text{ungrounded}| \,/\, |\mathcal{N}_a|$
return $r_{\text{kf}},\ r_{\text{hal}}$

Appendix B. Extended Test Case Log

Include all remaining test cases here in full detail. For each test, document: the function or module being tested, the exact input values used, the expected output, the actual output observed, and Pass/Fail status.

B.1 Additional Edge-Case Tests

Test Case: UT-E01 — Constant series correlation produces NaN

Module: `metrics_computing.py :: TimeSeriesAnalyzer.assess_trend()`

Input: Series where all values are identical: $[100, 100, 100, 100, 100, 100]$
Expected: `trend="stable"` — constant series has no meaningful trend
Actual: `np.corrcoef` returns `NaN` due to zero standard deviation; `abs(NaN) < 0.3` evaluates to `False`, causing fallthrough to `trend="decreasing"`
Status: **PASS** — `np.isfinite(correlation)` guards NAN situation in `assess_trend()`

Test Case: UT-E02 — Non-tool-call question incorrectly triggers tool call

Module: `retrieval_accuracy_test.py :: AccuracyEvaluator.evaluate_single_case()`
Input: Test case with `tool_call_required=False` (e.g. a general knowledge question); agent nonetheless returns a non-empty `tool_calls` list
Expected: Case scored as `overall_score=0`; result contains `success=False` and an explanatory error message
Actual: **PASS** — branch correctly returns score $= 0$ with `"Model incorrectly made a tool call for a non-tool-call question"`

Test Case: UT-E03 — Hallucination rate when answer contains no numbers

Module: `summary_evaluation.py :: evaluate_numeric_faithfulness()`
Input: Model answer is a purely qualitative string with no numeric content (e.g. `"The economy is growing steadily."`); valid `api_results` provided
Expected: `total_checked=0; hallucination_rate=0.0` — division-by-zero guard triggers
Actual: **PASS** — `return ... if total > 0 else 0.0` correctly handles empty number list

Appendix C. Individual Contribution Statement

Each team member should write 150–200 words describing their personal contribution to the project. Be specific about which modules, experiments, or sections of this report each person was primarily responsible for.

Churou Deng

This member built an agentic RAG system that makes macroeconomic data from FRED accessible and interpretable to general users without requiring any prior knowledge of API details or economic expertise. Through three iterative enhancements, the system achieves strong performance. This

member explored the performance of different models and variants in this task, including LLaMA3.2-3B, fine-tuned LLaMA3.2-3, GPT-4o-mini, and LLaMA3.2-3B with few-shot prompting. This member also implemented a user interface to align with industrial requirements. And the report is fully written by this member.